
Image-based waste-bin collection decisions

Marco Cesarato
EPICODE Institute of Technology
Computer Vision
18 July 2026

Abstract

Waste collection teams lose time when trucks visit bins that still have space. This project builds and tests a computer-vision pipeline that reads a bin photo and predicts whether the bin can wait or needs collection.

The dataset contains 5,654 JSON-annotated garbage-can images. The code maps each image to one of two labels: `has_space` or `needs_collection`. The deployed classifier uses ResNet50, a pretrained image model, and adapts it to this task over 5 training epochs. It reaches 0.8788 accuracy and 0.8596 macro F1 on a 1,130-image test split. Macro F1 gives both classes equal weight. For comparison only, an OpenCV HOG + Linear SVM experiment reaches 0.7062 accuracy and 0.6746 macro F1 on the same split. The API sends uncertain or risky CNN predictions to manual review. The main limits are dataset balance, split design, and missing labels for non-bin images.

1 Problem statement

1.1 Why the problem is relevant

City waste collection has a simple trade-off. Trucks that visit too early waste fuel, route time, and usable bin capacity. Bins left full can cause health problems and complaints. A photo check gives operators another signal when they decide which stops should come first.

The prototype in this report uses bin photos to help with that decision. It reads one image and predicts whether the visible bin still has room or needs collection.

The task has two possible answers: `has_space` and `needs_collection`.

1.2 Decision task and project scope

The intended use is narrow: collection recommendations for local city services. Operators should upload images that contain waste bins. Non-bin uploads are outside the current scope because the dataset does not include a rejection class.

2 Methodology: data, labels, and architecture

2.1 Dataset and target labels

The dataset contains 5,654 local garbage-can images with JSON labels. It includes 3,768 images tagged `is_empty` and 1,886 tagged `is_full`. The code maps those labels to the two decisions a collection team needs: `has_space` and `needs_collection`.

2.2 Label mapping

The dataset provides two fill-level tags:

- `is_full`: at least one visible bin is full or overflowing.
- `is_empty`: the bin still has free space. This can mean fully empty or partly filled.

The JSON labels match the final two-class task, but `is_empty` needs one note. In this dataset it means the bin can still accept waste, not always that it is completely empty. Table 1 gives the mapping used in the code and report.

Original tag	Final class
<code>is_empty</code>	<code>has_space</code>
<code>is_full</code>	<code>needs_collection</code>

Table 1: Label mapping.

The labels do not describe street conditions, loose waste around the bin, or non-bin images. For that reason, the model only learns the fill-level decision.

2.3 Held-out split

Before training, 20% of the images are set aside for testing. The split is stratified, so the train and test

sets keep a similar class balance. The model does not train on the test images. Table 2 lists the resulting counts.

Split	Images	has_space	needs_collection
Train	4,524	3,015	1,509
Test	1,130	753	377

Table 2: Stratified held-out train/test split.

2.4 Dataset considerations

The dataset has about twice as many `has_space` images as `needs_collection` images. The stratified split keeps that ratio in the test set, so the reported scores match the dataset balance.

The image order suggests repeated or related photo groups. A stronger evaluation should split by case block or photo session. This would test the model on new bins, locations, lighting conditions, and camera positions.

Because the data contains bin images only, the current system cannot reject non-bin uploads, unusual viewpoints, severe blur, night scenes, or blocked views. A real service would need labeled examples of those cases.

2.5 Image preprocessing

Preprocessing converts each uploaded image into the format expected by the model.

OpenCV reads the image, converts the color order, resizes it to 96×96 pixels, and scales pixel values from 0 to 1. The saved Keras model then applies the ResNet50 input steps. Training and API predictions use the same preprocessing steps.

2.6 ResNet50 transfer-learning architecture

The main model uses transfer learning with ResNet50 (He et al. 2016). ResNet50 is a neural network already trained on ImageNet, a large image dataset. Starting from those learned features helps because this project has only 5,654 images. The code keeps the ResNet50 feature-extraction layers fixed and trains a small final classifier for the two waste-bin labels.

The API uses the ResNet50 CNN because it has the highest macro F1 score in this project. Training runs for 5 epochs with batch size 32. The trained model is saved as `models/model.keras`.

The training split has 3,015 `has_space` images and 1,509 `needs_collection` images. Evaluation uses macro F1 because it gives both classes equal weight. This matters because missing the smaller class, `needs_collection`, is the riskier error.

3 Experimental results: tables, plots, and metrics

Evaluation uses the held-out test split. Accuracy is the share of correct predictions. Precision says how many

predicted positives are correct. Recall says how many real positives are found. Macro precision, macro recall, and macro F1 give equal weight to both classes. The confusion matrix breaks down the mistakes by class.

3.1 Held-out test results

Table 3 reports the main ResNet50 transfer-learning CNN results on the held-out split.

Metric	Value
Accuracy	0.8788
Macro Precision	0.8725
Macro Recall	0.8501
Macro F1	0.8596

Table 3: Stratified held-out test metrics for the main ResNet50 transfer-learning CNN.

3.2 Comparison experiment

The HOG + Linear SVM result is included only as a comparison experiment. It is not part of the API pipeline. The experiment checks whether a simpler course-style method can match the ResNet50 macro F1 score.

The source files are `src/compare/data.py`, which contains HOG extraction, split loading, and metric calculation, and `src/compare/models.py`, which runs the comparison evaluation command.

The HOG comparison converts each image to grayscale, extracts a 64×64 Histogram of Oriented Gradients descriptor (Dalal and Triggs 2005), and trains a balanced LinearSVC. HOG describes edge and shape patterns, so this comparison uses handcrafted features instead of CNN-learned features. The table reports results rerun on the 5,654-image dataset.

The HOG baseline follows the classical method shown in the course INRIA pedestrian-detection lab: grayscale images, OpenCV HOG descriptors, and a linear SVM classifier. The shared HOG layout uses 16×16 normalization blocks, 8×8 cells, an 8×8 block stride, and 9 orientation bins. These settings keep local edge detail, reduce lighting effects, and keep the feature vector small enough for a linear classifier.

The project-specific changes are in the data handling and evaluation. The implementation reads the project JSON labels, uses the project train/test split, maps labels to `has_space` and `needs_collection`, uses a square 64×64 window instead of the pedestrian-specific 64×128 window, applies class balancing, and reports macro metrics and a confusion matrix.

Table 4 compares test performance for the API CNN and the HOG experiment. Macro F1 is the main comparison metric because the dataset is imbalanced. A model cannot score well on macro F1 by mostly predicting the larger class.

Model	Input/features	Acc.	Macro Prec.	Macro Rec.	Macro F1
ResNet50 transfer CNN	RGB 96x96 pixels	0.8788	0.8725	0.8501	0.8596
OpenCV HOG 64x64 + Linear SVM	OpenCV HOG 64x64	0.7062	0.6724	0.6776	0.6746

Table 4: Main ResNet50 transfer-learning CNN and HOG + Linear SVM comparison experiment on the same stratified held-out split.

The 5-epoch CNN correctly predicts 705 `has_space` images and 288 `needs_collection` images. It also makes 48 false alarms and misses 89 full-bin cases. These counts give 0.8725 macro precision, 0.8501 macro recall, and 0.8596 macro F1.

On this test split, the CNN has higher accuracy and macro F1 than the HOG + Linear SVM comparison experiment. It improves macro F1 by 0.1850 and reduces missed full-bin cases from 154 to 89. The same comparison appears in Figure 1. The API uses only the CNN pipeline.

Table 5 gives the CNN confusion matrix on the held-out test split.

	<code>has_space</code>	<code>needs_collection</code>
<code>has_space</code>	705	48
<code>needs_collection</code>	89	288

Table 5: Confusion matrix on the stratified held-out split.

The CNN correctly predicts 288 of 377 full bins and 705 of 753 bins with space.

A naive model that always predicts `has_space` would still reach 66.6% accuracy because that class is larger, but it would miss every bin that needs collection. Accuracy alone is not enough for this task. Macro F1 fits the task because it scores each class separately and then averages the two scores.

For the CNN, the confusion matrix gives `has_space` F1 = 0.9114 and `needs_collection` F1 = 0.8079. Macro F1 is their average: $(0.9114 + 0.8079)/2 = 0.8596$.

3.3 Example predictions

Figure 2 contains four held-out predictions from the saved CNN: one correct `has_space` case, one correct `needs_collection` case, one mistake, and one prediction sent to manual review. The borderline example explains why some predictions should stay out of the automatic route.

4 Methodology: postprocessing and decision routing

Postprocessing starts after the model predicts the fill-level class. It converts class scores into a collection decision and records how many predictions stay automatic after the risk-score policy.

This step does not change the image or retrain the model. It only uses the output of the saved CNN.

4.1 Prediction outputs

The API serves the CNN model stored at `models/model.keras`. The confidence score is the model score for the predicted class. The API also keeps the scores for both classes.

These scores drive the decision. The policy reads `P(needs_collection)` and routes the result to `urgent`, `low`, or `review`.

4.2 Grad-CAM inspection

Grad-CAM reads the predicted class score, uses gradients from that score to weight convolutional feature maps from the ResNet backbone, and produces a heatmap of image regions that support the selected class.

This is a sanity check for model behavior. If the heatmap focused mainly on background objects, the model could be using a shortcut instead of the bin fill level.

Figure 3 shows one correct held-out example for each class. The heatmaps focus on bin and fill-level regions, which fits the intended task. These examples are visual checks only. They do not prove that the model is reliable in all cases.

4.3 Decision routing

The pipeline treats the `needs_collection` score as the risk score for the final decision. Fixed thresholds make each decision easy to check.

The policy also exposes a `risk_level`. It maps the `needs_collection` score to `high`, `medium`, or `low` risk for the API response.

The API applies one direct risk-score policy. If `P(needs_collection)` is at least 0.70, the bin is marked `urgent`. If `P(needs_collection)` is at most 0.15, the bin is marked `low`. Scores between 0.15 and 0.70 go to `review`. This keeps borderline full-bin risk visible to a human reviewer.

The API assigns the final collection priority as follows:

- If `P(needs_collection)` is at least 0.70, priority is `urgent`.
- If `P(needs_collection)` is at most 0.15, priority is `low`.
- If `P(needs_collection)` is between 0.15 and 0.70, priority is `review`.

The low-priority threshold is 0.15 and the urgent threshold is 0.70. The middle band is reviewed instead of being treated as an automatic decision.

4.4 Routing effect on the test split

On the test split, the risk-score policy sends 218 of 1,130 CNN predictions to review, or 19.3% of the test set. These are the predictions whose `needs_collection` risk score falls between 0.15 and 0.70. The remaining automatic predictions have 0.9298 accuracy, com-

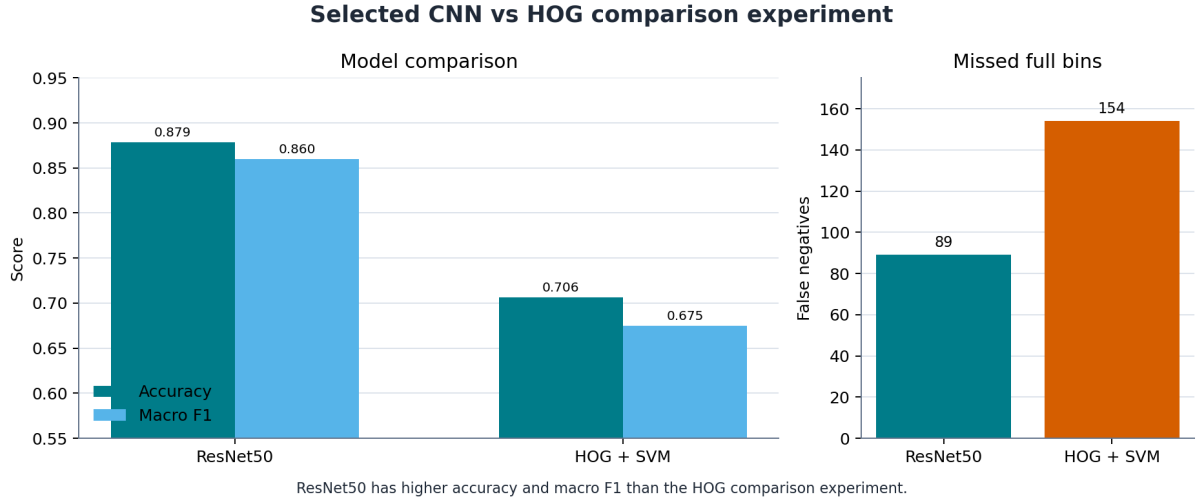


Figure 1: Model comparison between the selected ResNet50 CNN and the HOG + Linear SVM comparison experiment.

pared with 0.8788 accuracy across all CNN predictions. For the automatic cases, full-bin recall is the share of real full bins that are automatically marked `needs_collection`. Table 6 reports the routing effect.

Table 6: Postprocessing risk-score policy on the CNN held-out predictions.

Metric	Value
Urgent risk threshold	0.70
Low-priority risk threshold	0.15
Predictions sent to review	218
Review share	19.3%
Automatic decisions	912
Automatic urgent decisions	289
Automatic low-priority decisions	623
Accuracy on automatic predictions	0.9298
Full bins still missed automatically	42
Automatic full-bin recall	0.8641

Figure 4 traces how the risk-score policy converts class scores into review, urgent, or low-priority decisions.

The API response separates prediction details, collection priority, and request timing. `prediction` contains the label, confidence score, and class scores. `decision` contains the priority, risk level, and reason. `metadata` contains request processing time. The example below gives the public response structure.

5 Methodology: API deployment architecture

5.1 API and web interface

The web application in `src/app.py` loads the saved CNN model. FastAPI exposes the JSON prediction endpoint. Gradio supplies the browser upload page, which is mounted on the same FastAPI app at the application root.

Running Uvicorn from the project root starts both entry points. The Gradio page accepts a waste-bin image, shows a short prediction summary, and displays the raw API-style JSON response. The API accepts requests at `POST /predict`.

The browser page reuses the API prediction path. It receives the uploaded image as an RGB NumPy array, converts it to OpenCV BGR, and calls the same CNN prediction and risk-routing code as `/predict`. The label, confidence score, collection priority, risk level, reason, and processing time therefore stay consistent across the two interfaces.

The `/predict` endpoint accepts one uploaded image. It reads the image with OpenCV, prepares it for the CNN, predicts the fill-level class, and applies the risk-score policy. Table 7 lists the public response fields.

Table 7: Example API response fields for `POST /predict`.

Section	Field	Example value
prediction	label	needs_collection
	confidence	0.9812
prediction.scores	has_space	0.0188
	needs_collection	0.9812
decision	priority	urgent
	risk_level	high
decision	reason	Needs_collection risk 0.98 meets urgent threshold 0.70.
	processing_time_seconds	0.1234

The deployment uses these choices:

- The model loads once when the application starts, so requests do not reload it from disk.
- Image preparation and prediction run in a background thread pool.
- The API converts each uploaded image to RGB, resizes it to 96×96 , and normalizes pixel values before prediction.
- The saved CNN returns `has_space` or `needs_collection` for valid bin images.
- The browser interface lets an operator upload a bin image and inspect the prediction, confidence score, priority, processing time, and JSON output.

Held-out prediction examples



Figure 2: Held-out prediction examples from the saved ResNet50 model. Each panel reports the ground-truth label, predicted label, confidence score, and final routing priority.

6 Failure analysis: error cases and limits

The most important failure mode is a missed full bin: the CNN predicts **has_space** when the ground truth is **needs_collection**. On the full held-out split this happens 89 times. After the risk-score policy, 42 full-bin cases are still missed automatically. This is the riskiest error because it can leave an overflowing bin uncollected.

False alarms are less severe but still costly. The CNN predicts **needs_collection** for 48 bins that actually have space. This can send a truck to a bin that could have waited.

Likely causes include unclear fill levels, partly visible bins, waste bags near the rim, blocked views, lighting changes, camera angle differences, and clutter around the bin. The dataset also contains ordered image groups, so a random stratified split may place very similar cases in both train and test data. A stricter group-based split would better test whether the model works on new bins, locations, and photo sessions.

The current data contains bin images only. The model is not trained to reject non-bin uploads, severe blur, night scenes, blocked views, or unusual waste

around the bin. Those cases should go to manual review or be covered by extra training labels before use in a city service.

Next work should add group-based validation, non-bin examples, and more difficult **needs_collection** cases.

7 Ethical considerations: bias, privacy, and human oversight

The main bias risk is dataset coverage. If images come from only a few bins, streets, lighting conditions, seasons, or camera positions, the model may work better there than in other areas or with other bin types. That gap could make service quality uneven. A real service dataset should include many locations, weather types, bin designs, fill levels, and camera devices.

Privacy matters because street-level bin photos may accidentally include people, faces, house numbers, license plates, or private property. Images should be collected only when needed for the waste task. They should be stored for the shortest useful time and anonymized when possible. Public reports should avoid examples that show private details.

The system should support operators, not replace

ResNet50 Grad-CAM examples

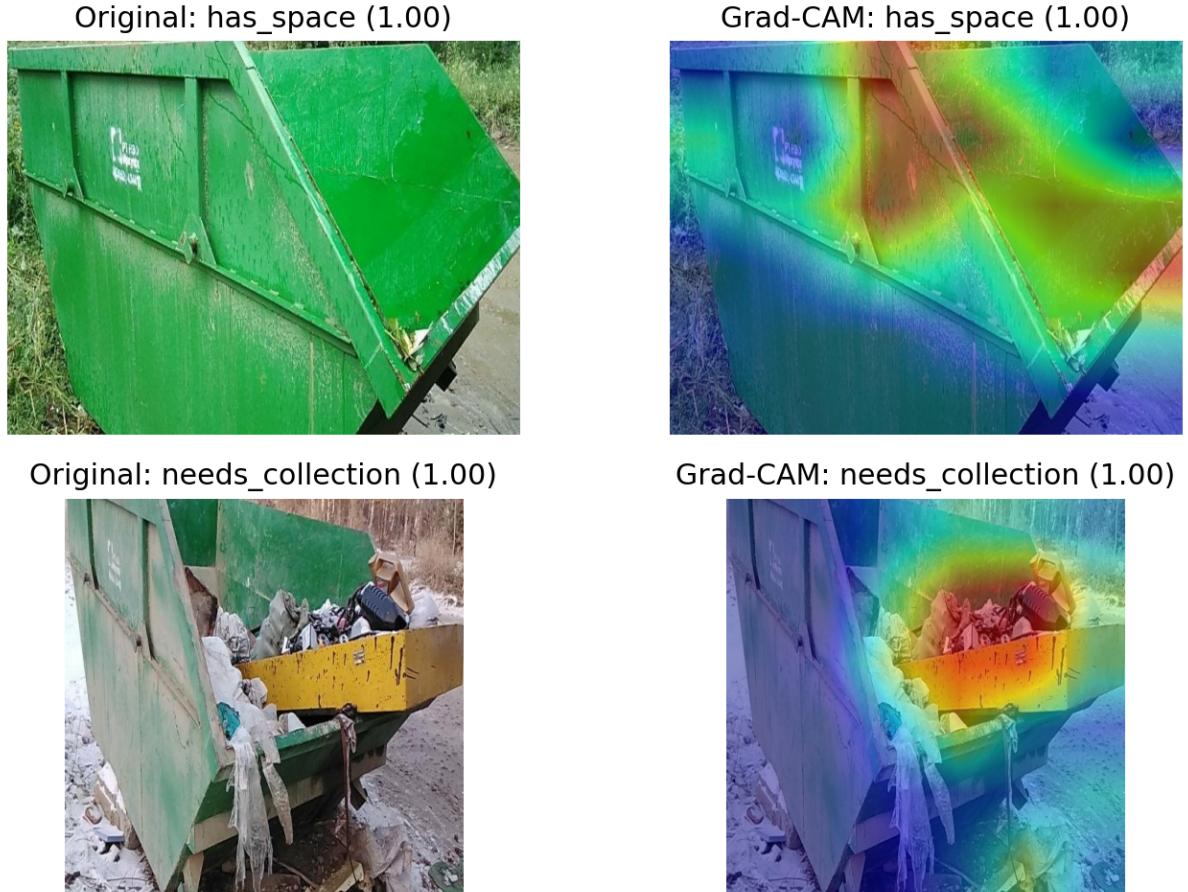


Figure 3: Grad-CAM examples for correct ResNet50 predictions on held-out images. Warmer colors show regions that contributed more to the selected class score.

them. Manual review is important for middle-band risk scores, unusual images, and complaints. The model should not be used for fines, surveillance, or individual profiling because the dataset and target labels only support a collection-priority decision.

8 Conclusion

This project turns waste-bin photos into a collection-priority decision. The model predicts **has_space** when a bin can wait and **needs_collection** when it should be collected. Training uses a ResNet50 image model and a stratified split with a 20% test set.

The test set contains 753 **has_space** and 377 **needs_collection** images.

The evaluation compares the ResNet50 CNN with a HOG + Linear SVM experiment. This comparison is not part of the API pipeline. It reaches 0.7062 accuracy and 0.6746 macro F1. It identifies 223 of 377 **needs_collection** images and 575 of 753 **has_space** images.

The main CNN uses ResNet50 with ImageNet weights and trains only the final classification layers for the two waste-bin classes. After 5 epochs, it reaches 0.8788 accuracy and 0.8596 macro F1. These scores are higher than the comparison experiment on this split, so

the API uses the CNN.

The dataset has clear limits. It contains about twice as many **has_space** examples as **needs_collection** examples. A stronger evaluation needs a group-based split so images from the same bin, location, or photo sequence do not appear in both train and test data.

The API adds a direct risk-score policy. If $P(\text{needs_collection})$ is at least 0.70, the decision is **urgent**. If it is at most 0.15, the decision is **low**. Scores between 0.15 and 0.70 go to **review**. On the test split, this sends 218 of 1,130 predictions to review and raises accuracy on the remaining automatic predictions from 0.8788 to 0.9298. The automatic route still misses 42 full-bin cases.

This risk-score policy does not add a new detector or depend on labels that the dataset does not have. It uses the existing class scores to make the final decision easier to inspect.

The web API assumes uploaded images contain waste bins. Operators should not use non-bin images as valid model inputs.

The pipeline is still a project test, not a live city system. The next version should add non-bin examples, harder full-bin examples, grouped images for stricter validation, and privacy checks for street-level photos.

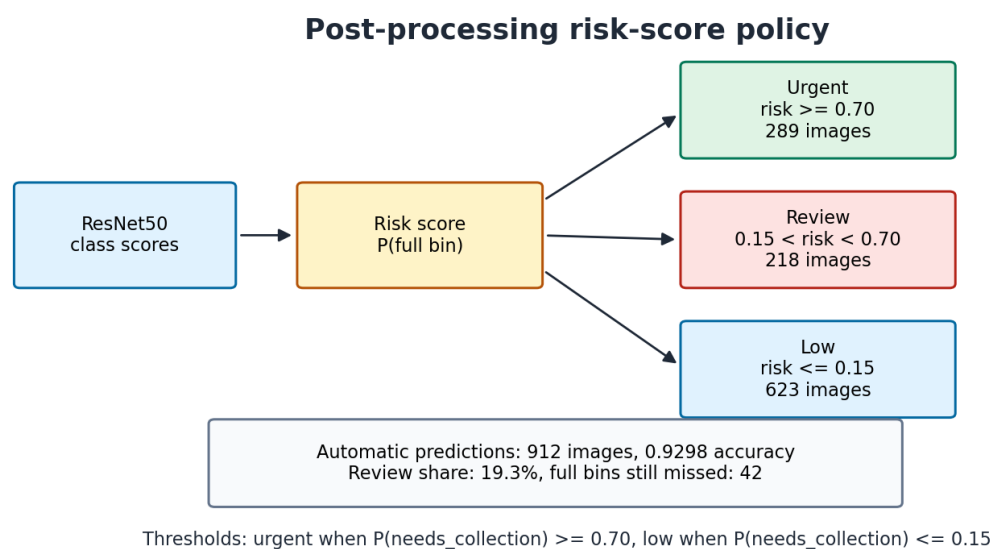


Figure 4: Postprocessing flow used to convert the ResNet50 `needs_collection` risk score into review, urgent, or low-priority decisions.

References

- He, Kaiming et al. (2016). “Deep Residual Learning for Image Recognition”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778. DOI: [10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90).
- Dalal, Navneet and Bill Triggs (2005). “Histograms of Oriented Gradients for Human Detection”. In: *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 886–893. DOI: [10.1109/CVPR.2005.177](https://doi.org/10.1109/CVPR.2005.177).